

```

1  (*
2      CS 51 Lab 19
3      Conway's Game of Life Cellular Automaton
4  *)
5  (*
6      SOLUTION
7  *)
8  module G = Graphics ;;
9
10 (* Automaton parameters *)
11 let cGRID_SIZE = 100 ;; (* width and height of grid in cells *)
12 let cSPARSITY = 5 ;; (* inverse of proportion of cells initially live *)
13 let cRANDOMNESS = 0.00001 ;; (* probability of randomly modifying a cell *)
14
15 (* Rendering parameters *)
16 let cCOLOR_LIVE = G.rgb 93 46 70 ;; (* color to depict live cells *)
17 let cCOLOR_DEAD = G.rgb 242 227 211 ;; (* background color *)
18 let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
19 let cSIDE = 8 ;; (* width and height of cells (in pixels) *)
20 let cRENDER_FREQUENCY = 1 (* how frequently grid is rendered (in ticks) *) ;;
21
22 (* Font specification for rendering the legend. OCaml font handling is
23    platform dependent, so we leave this as `None`, but we provide the
24    functionality for use in the solution set. Feel free to play with
25    this or leave it as is. *)
26 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
27
28
29 (*-----
30    The Game of Life
31    *)
32
33 (*.....
34    Game of Life states and updates
35    *)
36
37 (* We take the opportunity to abstract the cell states as an
38    enumerated type *)
39 type life_state =
40   | Live
41   | Dead ;;
42
43 (* flipped state -- Returns the opposite state to `state`. *)
44 let flipped (state : life_state) : life_state =
45   match state with
46   | Live -> Dead
47   | Dead -> Live ;;
48
49 (* offset index off -- Returns the `index` offset by `off` allowing
50    for wraparound. *)
51 let rec offset (index : int) (off : int) : int =
52   if index + off < 0 then

```

```

53     cGRID_SIZE - (offset ~-index ~-off)
54 else
55     (index + off) mod cGRID_SIZE ;;
56
57 (* life_update grid i j -- Returns the updated value for cell at `i,
58    j` in the grid based on rules of Conway's Game of Life. In this
59    implementation, after updating as per the standard GoL update rule,
60    a cell's state is then flipped with probability given by
61    cRANDOMNESS. *)
62 let life_update (grid : life_state array array) (i : int) (j : int)
63     : life_state =
64
65     (* We give a few variant update rules for the Game of Life. These
66        lists are indexed by adjacency count to give the generated cell
67        status.
68
69           0      1      2      3      4      5      6      7      8
70           |      |      |      |      |      |      |      |      |
71           v      v      v      v      v      v      v      v      v *)
72     (* B3/S23: Conway's original game of life *)
73     let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Dead; Dead; Dead] in
74     let live_update = [Dead; Dead; Live; Live; Dead; Dead; Dead; Dead; Dead] in
75
76     (* B3/S12345: https://conwaylife.com/wiki/OCA:Maze
77        let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Dead; Dead; Dead] in
78        let live_update = [Dead; Live; Live; Live; Live; Live; Dead; Dead; Dead] in
79
80     *)
81     (* B36/S125: https://conwaylife.com/wiki/OCA:2x2
82        let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Live; Dead; Dead] in
83        let live_update = [Dead; Live; Live; Dead; Dead; Live; Dead; Dead; Dead] in
84
85     *)
86     (* B3678/34678: https://conwaylife.com/wiki/OCA:Day\_%26\_Night
87        let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Live; Live; Live] in
88        let live_update = [Dead; Dead; Dead; Live; Live; Dead; Live; Live; Live] in
89
90     *)
91
92     let neighbors = ref 0 in
93     for i' = ~-1 to ~+1 do
94         for j' = ~-1 to ~+1 do
95             let i_ind = offset i i' in
96             let j_ind = offset j j' in
97             neighbors := !neighbors
98                 + match grid.(i_ind).(j_ind) with
99                     | Live -> 1
100                     | Dead -> 0
101
102         done
103     done;
104     (* determine new state based on neighbor count *)
105     let new_state =
106         match grid.(i).(j) with
107         | Live -> List.nth live_update (!neighbors - 1)
108         | Dead -> List.nth dead_update !neighbors in
109     (* randomly flip an occasional cell state *)
110     if Random.float 1.0 <= cRANDOMNESS then
111         flipped new_state

```

```

107     else
108         new_state ;;
109
110 (* life_initialize grid -- Fills `grid` with a random mix of Live and
111    Dead cells, with roughly 1-in-cSPARSITY cells initially live. *)
112 let life_initialize (grid : life_state array array) : unit =
113     for i = 0 to cGRID_SIZE - 1 do
114         for j = 0 to cGRID_SIZE - 1 do
115             grid.(i).(j) <- if Random.int cSPARSITY = 0 then Live else Dead
116         done
117     done
118
119 (* .....
120    Making the Game of Life
121    *)
122
123 (* GoL automaton specification *)
124
125 module LifeSpec : (Cellular.AUT_SPEC
126                    with type state = life_state) =
127     struct
128         type state = life_state
129         let initial : state = Dead
130         let grid_size : int = cGRID_SIZE
131         let initialize = life_initialize
132         let update = life_update
133         let name : string = "Conway's Life"
134         let cell_color (cell : state) : G.color =
135             match cell with
136             | Live -> cCOLOR_LIVE
137             | Dead -> cCOLOR_DEAD
138         let side_size : int = cSIDE
139         let legend_color : G.color = cCOLOR_LEGEND
140         let font : string option = cFONT
141         let render_frequency : int = cRENDER_FREQUENCY
142     end ;;
143
144 (* GoL automaton *)
145
146 module Aut = Cellular.Automaton (LifeSpec) ;;
147
148 (* .....
149    Running the automaton and displaying the results
150    *)
151
152 (* main seed -- Runs the automaton using the provided random `seed`
153    (for replicability), displaying updates to the graphics window. *)
154 let main seed =
155
156     Random.init seed;      (* initialize randomness *)
157     Aut.graphics_init ();  (* ... and graphics window *)
158
159     Aut.run_grid (Aut.initialized_grid ()) ;;
160

```

```
161 (* Run the automaton with user-supplied seed *)
162 let _ =
163   if Array.length Sys.argv <= 1 then
164     Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
165                   Sys.argv.(0)
166   else
167     main (int_of_string Sys.argv.(1)) ;;
168
```